Modulo 2

Actitud Positiva :)

2026-04-08

Introducción

Este documento contiene información sobre R Markdown. Algunos conceptos útiles, ejercicios aplicados, comparativas con otros lenguajes como Python, y demás cosas.

Markdown y R Markdown

Markdown es una sintaxis de texto simple para crear documentos estructurados utilizando texto plano. Util para la creación de documentos en formatos como HTML, PDF y Word, sin interfaces gráficas. Es uina herramienta sencilla, rápida y de amplia utilidad.

R Markdown es una extension del md que permite mezclar texto con código R y obtener resultados.

Resumen de diferencias básicas

Markdown	R Markdown
Solo texto formateado	Texto + código R
No ejecuta nada	Ejecuta código
Estático	Dinámico

¿Para qué sirve R Markdown?

En ingeniería es extremadamente útil para:

- Informes técnicos
- Análisis de datos
- Trabajos prácticos
- Automatización de resultados

R Markdown permite a otro correr tu archivo y obtener exactamente los mismos resultados

Uso de data sets o base de datos internos de R

Usando el comando `data()` en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo `$` despues del comando de informacion especifico que queremos.

Primeros comandos de R

R es un lenguaje muy parecido a Matlab que trabaja con matrices.

```
A <- 10
B <- 789
C = B - A
D = C * A
C
```

```
## [1] 779
```

```
D
```

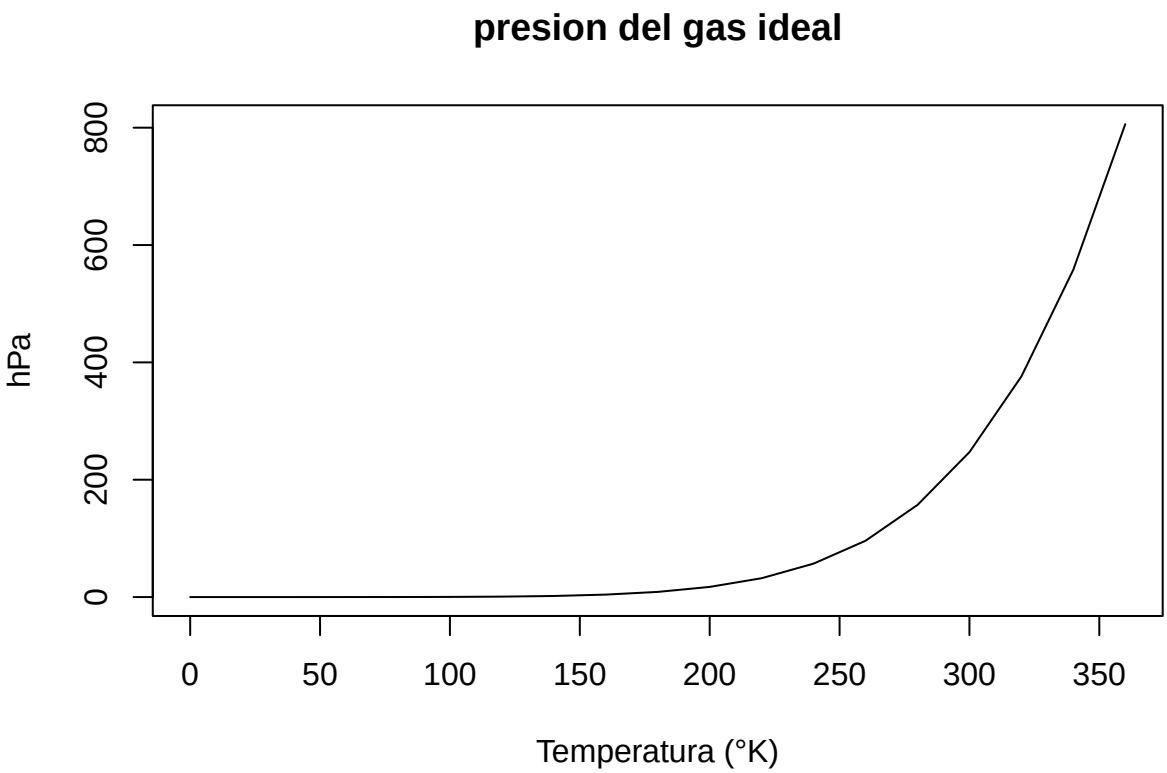
```
## [1] 7790
```

Comando Plot

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir `?plot` en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando.

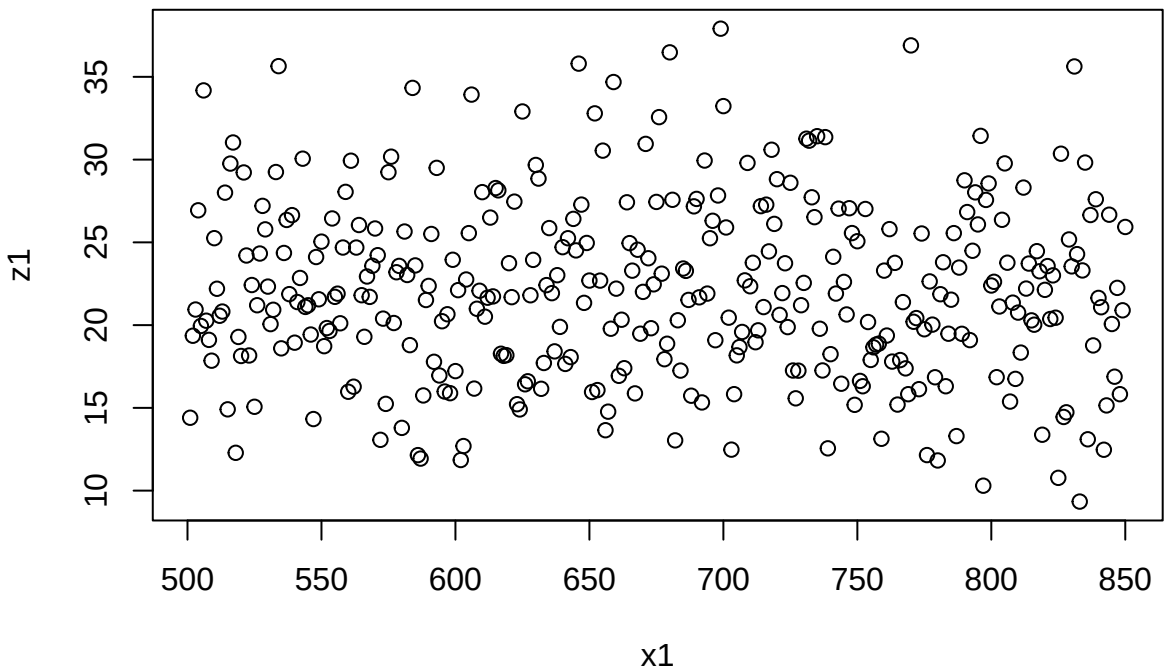
Comp ejemplo de aplicación tenemos:

```
plot(pressure,main="presion del gas ideal",xlab="Temperatura (°K)",ylab="hPa",type="l")
```

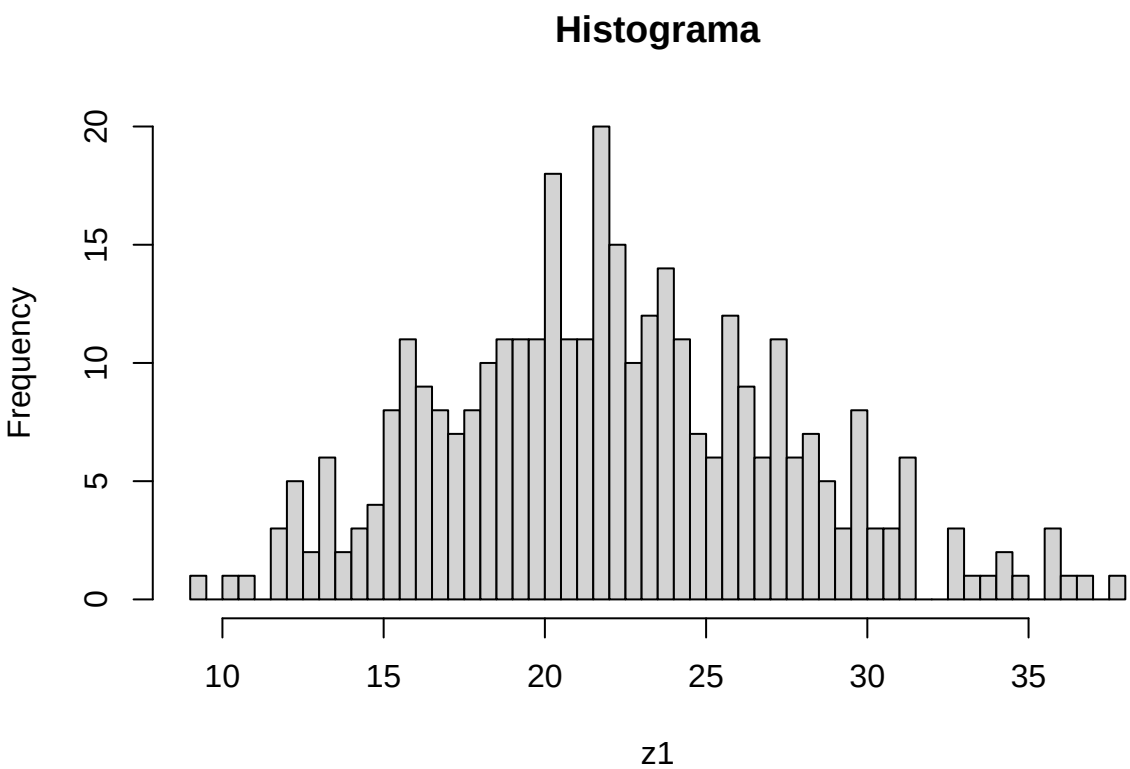


Funciones estadisticas

```
z1<-rnorm(350,22,5)
w1<-length(z1)
x1<-501:850
plot(x1,z1)
```




```r
hist(z1,main="Histograma",breaks=60)
```

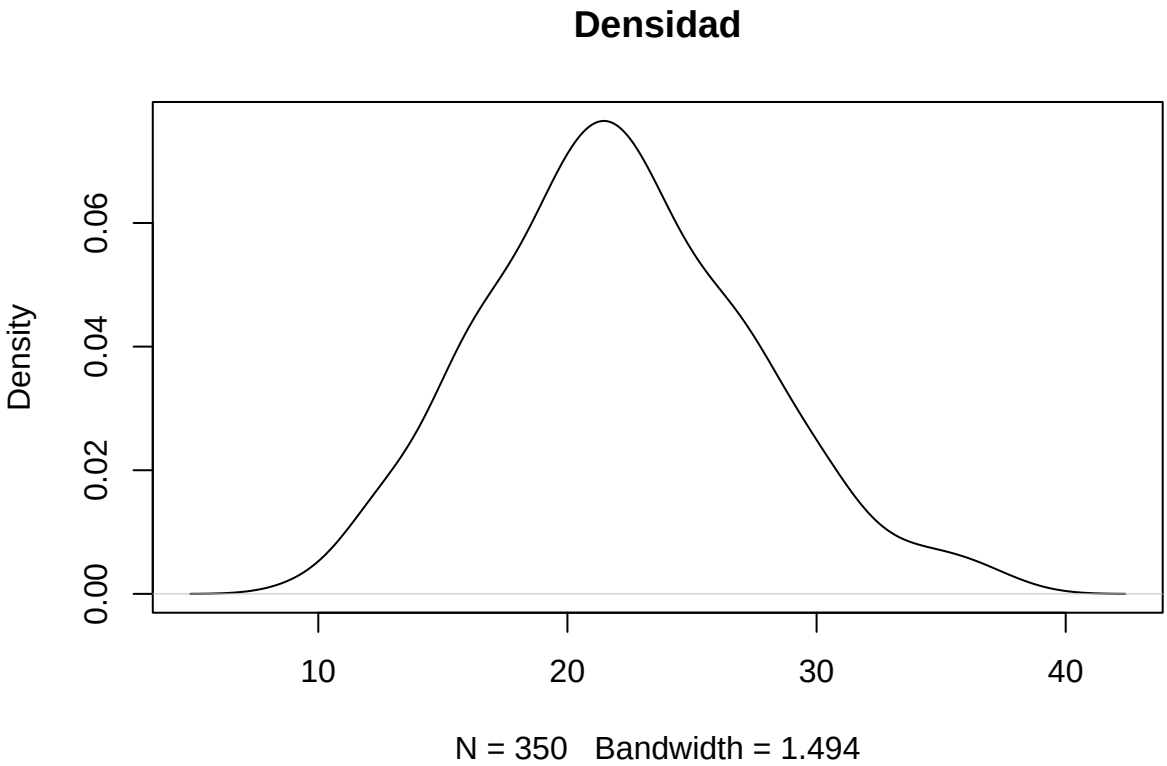



```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.494
```

```
##
##        x                y
##  Min.   : 4.857   Min.   :9.817e-06
##  1st Qu.:14.241   1st Qu.:3.386e-03
##  Median :23.624   Median :1.732e-02
##  Mean   :23.624   Mean   :2.659e-02
##  3rd Qu.:33.008   3rd Qu.:4.843e-02
##  Max.   :42.391   Max.   :7.651e-02
```

```
plot(density(z1),main="Densidad",type="l")
```



Desarrollo

Ejercicio 1:

Crear un vector que tenga todos los numeros enteros desde 1 a 762

```
secuencia_DNI<-(1:762)
secuencia_DNI
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
```

```
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762
```

Ejercicio 2:

Calcular la suma de todos los valores del vector secuencia_DNI

```r
total<-0
valor_final<-length(secuencia_DNI)
for (i in 1:valor_final){
  total<-total+i
}
total
```

```
## [1] 290703
```

Ejercicio 3

Repetir el ejercicio anterior pero en Python

```python

secuencia_DNI = list(range(1, 763))

suma_total = sum(secuencia_DNI)
```

```r
print(suma_total)
```

```
## 290703
```

Ejercicio 3.1:

¿Cuanto tarda en correr el código del ejercicio 2?

```r
inicio<-Sys.time()
total<-0
valor_final<-10*length(secuencia_DNI)
for (i in 1:valor_final){
  total<-total+i
}
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.00437355 secs
```

Ejercicio 3.2:

Repetir el ejercicio anterior usando la cabeza de Gauss.

```r
inicio <- Sys.time()
n <- 10 * length(secuencia_DNI)
total <- n * (n + 1) / 2
total
```

```
## [1] 29036010
```

```r
final <- Sys.time()
final - inicio
```

```
## Time difference of 0.001922607 secs
```

Ejercicio 4:

Generar una secuancia de 2 en 2 hasta 100000.

```r
system.time({
A<-numeric(50000)
for (i in 1:50000){
  A[i]<-i * 2
}
})
```

```
##    user  system elapsed
##   0.004   0.000   0.005
```

```r
  head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
  tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

Secuencia con funcion integrada de R:

```
system.time({
B<-seq(2,100000, by=2)
})
```

```
##    user  system elapsed
##       0       0       0
```

```
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(B)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

Ejercicio 5:

Implementación de una serie de Fibonacci.

Mediante función integrada:

¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
fib<- c(0,1)

while(tail(fib,1)<1e6) {
 fib <- c(fib, fib[length(fib)]+fib[length(fib)-1])
}
fib
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
print(paste("se requieren:",length(fib),"iteraciones"))
```

```
## [1] "se requieren: 32 iteraciones"
```

Ejercicio 6:

Implementación de una serie de Fibonacci.

Mediante definición matemática (método iterativo):

¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0 <- 0
f1 <- 1

contador <- 1

while (f1 <= 1000000) {
  f2 <- f1 + f0
  f0 <- f1
  f1 <- f2
```

```r
  contador <- contador + 1
}

print(paste("Iteraciones necesarias:", contador))
```

```
## [1] "Iteraciones necesarias: 31"
```

```r
print(paste("Valor alcanzado:", f1))
```

```
## [1] "Valor alcanzado: 1346269"
```

Ejercicio 7:

Ordenacion de un vector por método burbuja. Burbuja vs Sort

```r
library(microbenchmark)

x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)

  for (j in 1:(n - 1)){
    for (i in 1:(n - j)){
      if (x[i] > x[i + 1]){
        temp <- x[i]
        x[i] <- x[i + 1]
        x[i + 1] <- temp
      }
    }
  }
  return(x)
}

resultado <- microbenchmark(
  burbuja = burbuja(x),
  sort_R = sort(x),
  times = 3
)

summary(resultado)
```

```
##      expr         min           lq         mean       median           uq
## 1 burbuja 2.11328e+07 21155568.690 21240622.838 21178337.336 21294534.234
## 2  sort_R 2.14051e+02      231.706      361.305      249.361      434.932
##            max neval
## 1 21410731.133     3
## 2      620.503     3
```

Conclusión

Al comparar ambos métodos con microbenchmark, se observa que el método sort() posee una performance muy superior al método burbuja. Esto se debe a que el algoritmo burbuja compara pares de elementos repetitivamente, mientras que sort() utiliza algoritmos internos más eficientes. Para una muestra de tamaño 20.000, la diferencia de tiempo de ejecución es muy significativa, mostrando que el método burbuja resulta

poco adecuado para vectores grandes.

Ejercicio 8 - 1er Forma con Microbenchmark:

Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```
suma_iterativa <- function(n) {
  total <- 0
  for (i in 1:n) {
    total <- total + i
  }
  return(total)
}

suma_gauss <- function(n) {
  return(n * (n + 1) / 2)
}

library(microbenchmark)

n <- 1e6

resultado <- microbenchmark(
  iterativo = suma_iterativa(n),
  gauss = suma_gauss(n),
  times = 5
)

resultado
```

```
## Unit: nanoseconds
##       expr      min       lq       mean   median       uq      max neval
##  iterativo 15177219 15225790 15902186.8 15236470 15251619 18619836     5
##      gauss      540      910   403559.8     2260    12840  2001249     5
```

```
summary(resultado)
```

```
##        expr      min       lq       mean   median       uq      max neval
## 1 iterativo 15177219 15225790 15902186.8 15236470 15251619 18619836     5
## 2     gauss      540      910   403559.8     2260    12840  2001249     5
```

```
suma_iterativa(n)
```

```
## [1] 500000500000
```

```
suma_gauss(n)
```

```
## [1] 500000500000
```

Ejercicio 8 - 2da Forma

Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```
n <- 1e6
```

```r
system.time({
  suma <- 0
  for (i in 1:n) {
    suma <- suma + i
  }
})
```

```
##    user  system elapsed
##   0.018   0.000   0.017
```

```r
suma
```

```
## [1] 500000500000
```

```r
system.time({
suma2<-n * (n+1)/2
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
suma2
```

```
## [1] 500000500000
```

Conclusión

El método iterativo presenta mayor demora, por lo tanto menor eficiencia, debido a que realiza una suma por cada elemento. En cambio, el método de Gauss tiene menor demora, por tanto mayor eficiencia, ya que utiliza una fórmula directa que disminuye las iteraciones, recursos y por tanto los tiempos.

Esto hace que el método de Gauss sea significativamente más eficiente, especialmente para valores grandes de n. Para $n = 1 \times 10^{10}$, el método iterativo resulta impracticable, mientras que el método de Gauss obtiene el resultado de forma instantánea.

Sin embargo, por cuestiones de recursos disponibles para ejecutar y depurar los códigos se tomó como criterio $n = 1 \times 10^{6}$, debido a que un exponente 10 en el metodo iterativo colapsaba el entorno e imposibilitaba la ejecución